

Punto 2.6 - Configuración mediante Data Annotations

Ejercicio guiado: expresar mediante Data Annotations una parte del mapeo ya construido en AceriaData y comprobar cómo conviven los atributos con la Fluent API existente. El objetivo no es rehacer el modelo, sino aprender qué configuración puede viajar junto a las clases, qué configuración debe seguir en OnModelCreating y cuál prevalece cuando ambas expresan la misma regla.

Contexto del proyecto: 2.6 parte del estado completo de 2.5. Ya existen relaciones 1:N, 1:1 y la entidad intermedia OrdenAleacion con clave compuesta. En este punto se añaden atributos a las entidades acumulativas. Las relaciones complejas siguen configurándose con Fluent API. La migración M2_2_6 muestra además una consecuencia importante: mover reglas desde Fluent API a atributos puede cambiar metadatos del modelo, por ejemplo valores por defecto que antes estaban definidos en SQL.

Objetivos de aprendizaje

- Importar y utilizar System.ComponentModel.DataAnnotations y System.ComponentModel.DataAnnotations.Schema.
- Aplicar [Table], [Key], [Required] y [MaxLength].
- Aplicar [ForeignKey] en navegaciones.
- Aplicar [Precision] sobre propiedades decimal.
- Definir la PK compuesta de OrdenAleacion mediante [PrimaryKey] en EF Core 8.
- Comprender por qué dos [Key] no representan correctamente una PK compuesta en este escenario.
- Mantener relaciones complejas en Fluent API.
- Comprender la prioridad: Fluent API > Data Annotations > convenciones.
- Interpretar la migración real M2_2_6 y los defaults que desaparecen.
- Inspeccionar el modelo efectivo construido por EF Core.
- Comparar qué reglas proceden de atributos y cuáles quedan fijadas por Fluent API.

Paso 1: Abrir el estado autónomo de 2.6

```
cd M02/PROYECTO/2.6
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo de 2.6. Para reproducir la evolución manualmente, parte de una copia de 2.5 y aplica los cambios descritos a continuación.

Paso 2: Añadir los espacios de nombres necesarios

Al principio de Program.cs aparecen:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
```

Los dos primeros espacios de nombres proporcionan atributos como Key, Required, MaxLength, Table y ForeignKey. Los atributos específicos de EF Core, como Precision y PrimaryKey, están disponibles a través de Microsoft.EntityFrameworkCore.

Si falta alguno de estos using, el compilador no podrá resolver los atributos correspondientes.

Paso 3: Aplicar atributos a OrdenFabricacion

```
[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;

    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;

    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }

    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";

    [MaxLength(500)]
    public string? Observaciones { get; set; }
}
```

Table fija el nombre de tabla. Key identifica la PK simple. Required expresa obligatoriedad en propiedades de referencia y MaxLength fija el tamaño máximo de las columnas de texto. Observaciones continúa siendo nullable porque su tipo es string?.

No se añaden aquí índices, columnas calculadas ni propiedades nuevas: esos conceptos quedan fuera del estado ejecutable de 2.6.

Paso 4: Aplicar Table, Key, Precision y ForeignKey a PlanchaAcero

```
[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }

    public int OrdenId { get; set; }

    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }

    [Precision(18, 3)]
    public decimal Peso { get; set; }

    public bool Activa { get; set; } = true;

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

Precision se aplica a Peso, que es decimal. ForeignKey vincula la navegación Orden con la propiedad OrdenId. La cardinalidad, DeleteBehavior.Cascade y la obligatoriedad de la relación permanecen en Fluent API.

Paso 5: Aplicar atributos a Aleacion y EstadoOrden

```
[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;

    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;

    [MaxLength(500)]
    public string? Descripcion { get; set; }
}
```

EstadoOrden sigue el mismo patrón:

```
[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;

    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;

    public bool Activo { get; set; } = true;
}
```

Un tipo valor no anulable como bool ya es requerido por su propio tipo CLR; no necesita Required.

Paso 6: Aplicar atributos a DetalleOrden y CertificadoCalidad

```
[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }

    public int OrdenId { get; set; }

    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;

    [MaxLength(500)]
    public string? Notas { get; set; }

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

Y para CertificadoCalidad:

```
[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }

    public int OrdenId { get; set; }

    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;

    public DateTime FechaEmision { get; set; }

    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

Los atributos permiten identificar propiedades y FK, pero la cardinalidad 1:1 y el índice único derivado continúan definidos por la relación Fluent API heredada.

Paso 7: Declarar correctamente la clave compuesta de OrdenAleacion

```
[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }

    public DateTime FechaAsignacion { get; set; } = DateTime.Now;

    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }

    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";

    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}
```

No coloques dos atributos [Key], uno en cada FK. En EF Core 8 la forma declarativa adecuada para esta PK compuesta es PrimaryKey sobre la clase. La Fluent API conserva también HasKey con las dos propiedades. Si hubiese una contradicción, prevalecería Fluent API.

Paso 8: Mantener en Fluent API las relaciones y reglas complejas

La relación entre plancha y orden continúa configurada con:

```
entity.HasOne(x => x.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(x => x.OrdenId)
    .OnDelete(DeleteBehavior.Cascade)
    .IsRequired();
```

Las relaciones 1:1 siguen usando WithOne y HasForeignKey, y OrdenAleacion mantiene dos relaciones 1:N con distintos comportamientos de borrado:

```
entity.HasOne(x => x.Orden)
    .WithMany(o => o.OrdenesAleaciones)
    .HasForeignKey(x => x.OrdenFabricacionId)
    .OnDelete(DeleteBehavior.Cascade);

entity.HasOne(x => x.Aleacion)
    .WithMany(a => a.OrdenesAleaciones)
    .HasForeignKey(x => x.AleacionId)
    .OnDelete(DeleteBehavior.Restrict);
```

Esta combinación muestra el propósito de ambos mecanismos: atributos para reglas locales y Fluent API para relaciones o configuraciones que requieren más control.

Paso 9: Comprender la prioridad del modelo

EF Core aplica las fuentes de configuración con esta prioridad:

```
Convenciones < Data Annotations < Fluent API
```

Table("OrdenesFabricacion") y ToTable("OrdenesFabricacion") coinciden, por lo que no existe conflicto. La PK compuesta también aparece con PrimaryKey y HasKey y ambas expresiones describen la misma clave.

Si un MaxLength de un atributo y un HasMaxLength Fluent estableciesen valores distintos para la misma propiedad, el valor Fluent formaría parte del modelo final.

Paso 10: Compilar antes de generar la migración

```
dotnet build AceriaData.sln --configuration Release
```

El build debe finalizar correctamente antes de pedir a EF Core que compare el modelo actual con el snapshot de 2.5.

Paso 11: Generar M2_2_6 desde el estado 2.5

```
dotnet ef migrations add M2_2_6
```

El checkpoint ya contiene la migración real:

```
Migrations/20260927204758_M2_2_6.cs
```

No vuelvas a crear una migración con el mismo nombre dentro del estado final.

Paso 12: Revisar qué cambia realmente en M2_2_6

La migración no está vacía. EF Core detecta que varios defaults definidos antes mediante Fluent API ya no están presentes en el modelo 2.6.

FechaCreacion deja de tener el default SQL GETDATE(). También desaparecen defaults heredados de:

- OrdenesFabricacion.Estado, que tenía "Pendiente";
- PlanchasAcero.Activa, que tenía true;
- OrdenesAleaciones.FechaAsignacion, que tenía GETDATE();
- OrdenesAleaciones.EstadoRelacion, que tenía "Activa";
- EstadosOrden.Activo, que tenía true.

Esto demuestra que un inicializador de propiedad en C# no equivale a un DEFAULT definido en SQL Server. Dos configuraciones que parecen similares al leer la clase pueden producir metadata relacional distinto.

Paso 13: Aplicar y listar las migraciones

```
dotnet ef database update
dotnet ef migrations list
```

La lista debe incluir M2_2_6. Después de aplicar el historial, la base de datos refleja el snapshot del punto.

Paso 14: Ejecutar e inspeccionar el modelo efectivo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El estado final consulta context.Model y muestra:

```
2.6 OK | Tabla por annotations: OrdenesFabricacion | NumeroOrden MaxLength: 50
```

La salida observa el modelo efectivo que EF Core construyó después de combinar convenciones, atributos y Fluent API.

Paso 15: Resolver el reto Annotations vs Fluent API

Reto: explicar qué parte de la configuración procede de atributos y qué parte sigue controlada por Fluent API, comprobando el modelo resultante.

El checkpoint contiene el bloque comentado RETO 2.6 - ANNOTATIONS VS FLUENT API. Al activarlo muestra las categorías de configuración y los valores efectivos del modelo:

```
Annotations: Table, Key, Required, MaxLength, ForeignKey, Precision y PrimaryKey
Fluent API: ToTable, HasKey y relaciones explícitas tienen prioridad
Modelo efectivo OrdenesFabricacion | Tabla: OrdenesFabricacion
Modelo efectivo NumeroOrden MaxLength: 50
```

El reto no añade entidades ni cambia el esquema. Su finalidad es aprender a razonar sobre el origen de una configuración y el modelo efectivo que termina usando EF Core.

Errores comunes

Error	Causa	Corrección
Se colocan dos [Key] para OrdenAleacion	Se intenta modelar una PK compuesta como dos PK simples	Usar PrimaryKey con OrdenFabricacionId y AleacionId
Un atributo parece no tener efecto	Fluent API configura después la misma característica	Recordar Fluent API > annotations > convenciones
Se usa Precision esperando cambiar un double a decimal	El tipo CLR sigue siendo double	Aplicar Precision a propiedades decimal o cambiar conscientemente el tipo
Se confunde inicializador C# con default SQL	El valor C# sólo actúa al crear el objeto en .NET	Revisar la migración y distinguir valor CLR de DEFAULT SQL
Se elimina Fluent API de las relaciones	Los atributos no expresan todo el comportamiento necesario	Mantener HasOne, WithMany, WithOne, HasForeignKey y DeleteBehavior

Error	Causa	Corrección
Se introduce una entidad auxiliar nueva	Se amplía el dominio fuera del alcance del punto	Mantener 2.6 sobre las entidades acumuladas de 2.5
Se usa EnsureCreated()	Se evita el historial incremental	Mantener Migrations y Database.Migrate() / database update

Analugía final

Las Data Annotations son etiquetas colocadas directamente sobre las piezas del modelo: permiten ver de inmediato que una propiedad es obligatoria, tiene una longitud máxima o forma parte de una clave. La Fluent API se parece al libro de especificaciones de la planta: permite describir reglas que afectan a varias piezas y relaciones completas. Si la etiqueta y el libro discrepan, el libro de especificaciones tiene prioridad.

Resultado esperado

Al finalizar 2.6, el proyecto conserva todo el modelo de 2.5 y añade Data Annotations a las entidades existentes. OrdenAleacion mantiene su PK compuesta mediante PrimaryKey, las relaciones siguen gobernadas por Fluent API y M2_2_6 registra las diferencias reales del modelo, incluida la retirada de varios defaults SQL heredados. La ejecución termina con 2.6 OK y el reto permite explicar la prioridad entre atributos y Fluent API.

Conexión con el siguiente punto

2.7 continúa desde este mismo modelo y profundiza en Fluent API. Allí se estudiarán configuraciones que requieren un control centralizado mayor que el que ofrecen los atributos.

Código acumulativo completo del estado 2.6

El siguiente archivo es el Program.cs real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

```

}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]

```

```

    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
        });
        modelBuilder.Entity<DetalleOrden>(entity =>
        {
            entity.ToTable("DetallesOrden");
            entity.HasKey(d => d.Id);
            entity.HasOne(d => d.Orden)
                .WithOne(o => o.Detalle)
                .HasForeignKey<DetalleOrden>(d => d.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
    
```

```

        .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
        _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
    }
}

```

```

        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    }
                ))
            .LogTo(global::System.Console.WriteLine, new[] {
                "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
            .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
                FechaEmision = DateTime.Today,
            }
        }
    }
}

```

```

        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.6 OK | Tabla por annotations: {entity.GetTableName()} |
NumeroOrden MaxLength: {entity.FindProperty(nameof(OrdenFabricacion.NumeroOrden)).GetMaxLength()}");
}
}

```

Explicación línea a línea del código acumulativo 2.6

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.6/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo del ejercicio.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo del ejercicio.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad NumeroCertificado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad FechaEmision de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad OrganismoCertificador de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula explícitamente la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta con las propiedades indicadas.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase OrdenAleacion que forma parte del estado acumulativo del ejercicio.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad OrdenFabricacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad AleacionId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad FechaAsignacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad CantidadUtilizada de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad EstadoRelacion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad Aleacion de tipo Aleacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo del ejercicio.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 149: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 150: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 151: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 152: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 153: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 154: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 155: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 156: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 157: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 158: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 159: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 160: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 161: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 162: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 163: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 164: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 165: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 166: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 167: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 168: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 169: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 170: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 171: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 172: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 173: `entity.HasKey(d => d.Id);` → define la clave primaria de la entidad.

Línea 174: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 175: `.WithOne(o => o.Detalle)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 176: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 177: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 178: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 179: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 181: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 182: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 183: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 184: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 185: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 186: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 187: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 188: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 189: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 190: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 191: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 192: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 193: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 194: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria de la entidad.

Línea 195: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 196: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 197: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 198: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 199: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 200: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 201: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 202: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 203: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 204: `}` → cierra el bloque de código actual.

Línea 205: `}` → cierra el bloque de código actual.

Línea 207: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 208: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 209: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 210: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 211: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 212: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 213: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 214: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 215: `}` → cierra el bloque de código actual.

Línea 217: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 218: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 219: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 220: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 222: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 223: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 224: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null` cuando no existe.

Línea 225: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 226: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 227: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 228: `}` → cierra el bloque de código actual.

Línea 230: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 231: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 232: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 233: `}` → cierra el bloque de código actual.

Línea 235: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 236: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 237: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 238: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 239: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 240: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 241: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 242: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 243: `}` → cierra el bloque de código actual.

Línea 245: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 246: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 247: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 248: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 249: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 250: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 251: `}` → cierra el bloque de código actual.

Línea 252: `}` → cierra el bloque de código actual.

Línea 254: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 255: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 256: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 257: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 258: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 259: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 260: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 261: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 262: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 264: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 265: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 267: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 268: `services.AddDbContext<AceriaDbContext>(options => → registra el DbContext en el contenedor de inyección de dependencias.`

Línea 269: `options → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.`

Línea 270: `.UseSqlServer(connectionString, sql => → selecciona SQL Server como proveedor de EF Core.`

Línea 271: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 272: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null); → habilita reintentos ante fallos transitorios del proveedor SQL Server.`

Línea 273: `sql.CommandTimeout(60); → establece el tiempo máximo permitido para los comandos SQL.`

Línea 274: `}) → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.`

Línea 275: `.LogTo(global::System.Console.WriteLine, new[]
{ "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information) → configura la salida del logging de EF Core.`

Línea 276: `.EnableDetailedErrors(); → activa errores detallados para facilitar el diagnóstico del laboratorio.`

Línea 278: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.`

Línea 279: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>(); → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.`

Línea 281: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.`

Línea 282: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 283: `ValidateScopes = true, → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.`

Línea 284: `ValidateOnBuild = true → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.`

Línea 285: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 287: `using var scope = provider.CreateScope(); → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.`

Línea 288: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>(); → resuelve un servicio obligatorio desde el contenedor de dependencias.`

Línea 289: `context.Database.EnsureDeleted(); → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.`

Línea 290: `context.Database.Migrate(); → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.`

Línea 292: `var orden = new OrdenFabricacion → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.`

Línea 293: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 294: `NumeroOrden = "0F-M2-0001", → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.`

Línea 295: `Cliente = "Constructora del Norte"`, → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 296: `FechaCreacion = DateTime.UtcNow`, → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 297: `FechaEntrega = DateTime.Today.AddDays(14)`, → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 298: `Estado = "Pendiente"`, → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 299: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 300: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 301: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 302: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 303: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 304: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 305: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 306: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 307: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 308: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 309: `}` → cierra el bloque de código actual.

Línea 310: `}`, → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 311: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 312: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 313: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%"`, → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 314: `TemperaturaColada = 1550.5`, → asigna la temperatura de colada del detalle de fabricación.

Línea 315: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 316: `}`, → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 317: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 318: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 319: `NumeroCertificado = "CERT-0001"`, → asigna el identificador natural del certificado de calidad.

Línea 320: `FechaEmision = DateTime.Today`, → asigna la fecha de emisión del certificado usado en la prueba.

Línea 321: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 322: `}`, → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 323: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 325: `var aleacion = new Aleacion` → crea la aleación que se enlazará con la orden mediante la entidad intermedia `OrdenAleacion`.

Línea 326: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 327: `Nombre = "AISI 1045"`, → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 328: `Codigo = "A1045"`, → asigna el código natural único de la aleación.

Línea 329: `PorcentajeCarbono = 0.45`, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 330: `PorcentajeManganeso = 0.75`, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 331: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 332: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 333: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 334: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 335: `Aleacion = aleacion`, → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 336: `CantidadUtilizada = 1500.500m`, → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 337: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 338: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 339: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 340: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 341: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 343: `var entity = context.Model.FindEntityType(typeof(OrdenFabricacion))!;` → obtiene los metadatos EF Core de `OrdenFabricacion` para inspeccionar claves, índices o filtros del modelo construido.

Línea 344: `global::System.Console.WriteLine($"2.6 OK | Tabla por annotations: {entity.GetTableName()} | NumeroOrden MaxLength: {entity.FindProperty(nameof(OrdenFabricacion.NumeroOrden))?.GetMaxLength()}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 345: `}` → cierra el bloque de código actual.

Línea 346: `}` → cierra el bloque de código actual.